The problem of concurrency control occurs when the customer performs transaction T3 first, then another user runs T4, which could lead to a phantom read issue. Since T3 reads a database element whose delete action committed by T4 first. Here is the screen shot to depict the scenario.

Scenario 1: There is only one T3 performs.

```
-- Test transtraction T3
exec DisplaySaleReport @year = 2005, @month = 5;
```

Result:

```
SALE REPORT IN MONTH 5/2005
Number of invoices: 128
Number of products: 3909
Total revenue (factory price): 220051
Total retail revenue: 360331
Total profit: 140280
Status: {Profit}
Top 1 Best-selling product of the month: 1992 Ferrari 360 Spider red
Top 1 Slowest-selling product of the month:1968 Dodge Charger
```

Scenario 2: T3 performs, then T4 comes next, which cause a concurrency issue.

```
-- Test transtraction T3
exec DisplaySaleReport @year = 2005, @month = 5;

exec DeleteProductFromInvoice @order_number = 10419, @product_code = S24_3856
```

Result:

```
SALE REPORT IN MONTH 5/2005
Number of invoices: 127
Number of products: 3640
Total revenue (factory price): 203934
Total retail revenue: 331021
Total profit: 127087
Status: {Profit}
Top 1 Best-selling product of the month: 1992 Ferrari 360 Spider red
Top 1 Slowest-selling product of the month:1968 Dodge Charger
```

The result looks quite different from the previous one. Since there are some invoices has been deleted.

To handle this problem. I would attach the isolation level 3 (repeated read) to procedure DisplaySaleReport and implement UPLOCK.

Result:

After first execution of T3,T4

```
SALE REPORT IN MONTH 5/2005
Number of invoices: 125
Number of products: 2997
Total revenue (factory price): 172437
Total retail revenue: 280172
Total profit: 107735
Status: {Profit}
Top 1 Best-selling product of the month: 1992 Ferrari 360 Spider red
Top 1 Slowest-selling product of the month:1968 Dodge Charger
```

After second execution of T3

```
SALE REPORT IN MONTH 5/2005
Number of invoices: 124
Number of products: 2921
Total revenue (factory price): 168739
Total retail revenue: 274322
Total profit: 105583
Status: {Profit}
Top 1 Best-selling product of the month: 1992 Ferrari 360 Spider red
Top 1 Slowest-selling product of the month:1968 Dodge Charger
```

The problem of concurrency control occurs when the customer performs transaction T4 first, then another user runs T3, which could lead to a phantom read issue. Since T3 reads a database element whose delete action committed by T4 first. Similar to the first problem. I will implement isolation level 3 for T4 and implement UPLOCK to ensure that no action will be performed on the data which is locked earlier.

```
-- Test transaction T4
exec DeleteProductFromInvoice @order_number = 10425, @product_code = S50_1392
```

```
exec DisplaySaleReport @year = 2005, @month = 5;
```

When run T4 before T3. T3 will wait for T4 and display the result:

```
SALE REPORT IN MONTH 5/2005
Number of invoices: 128
Number of products: 3909
Total revenue (factory price): 220051
Total retail revenue: 360331
Total profit: 140280
Status: {Profit}
Top 1 Best-selling product of the month: 1992 Ferrari 360 Spider red
Top 1 Slowest-selling product of the month:1968 Dodge Charger
```